

VIET NAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF SCIENCE



INTRODUCTION TO BIG DATA ANALYSIS

Lecturer

PhD. Nguyen Ngoc Thao | nnthao@fit.hcmus.edu.vn
PhD. Le Ngoc Thanh | lnthanh@fit.hcmus.edu.vn

Lab Instructors

Tran Huy Ban | huyban.han@gmail.com
Huynh Lam Hai Dang | hlhdang@fit.hcmus.edu.vn

Lab 02: Advanced MapReduce & Spark Structured APIs

1. Preliminaries

- To do this lab, you need to know the basic concepts of Spark and Structured APIs. You can read the basics in this article ([Link](#)) or the official tutorial from Apache ([Link](#) – written in Python but it is similar in other languages).

1.1. The history of Spark

- In the early days of Spark, data operations were performed directly on RDDs, which were much harder to write and optimize.
- With the advent of DataFrame and Structured APIs, writing Spark jobs has become much easier and more efficient thanks to automatic optimization features.
- To fully and accurately understand all the concepts that power Spark like Catalyst optimizer, Whole-stage Code Gen, etc. I recommend you to read more in the book named ***Spark: The Definitive Guide***. (it is not required, but you should take a look).
- Later, more and more features were introduced, such as Adaptive Query Execution (AQE) introduced in version 3.0 to optimize Spark jobs, making them run significantly faster *in most cases*.

1.2. DataFrame

DataFrame in Spark has the following properties:

- ***Schema-based***: Dataframes have schemas, which require declaring the names and data types of the columns in the dataframe (that's why they are named Structured APIs). Supports many primitive data types as well as other complex data types.
- ***Immutable***: Dataframes in Spark cannot be modified after being created. However, we can transform them into another dataframe through **transformations**.
- ***Lazy evaluation***: When the user has not performed an **action**, the previous transformations (in the same stage) have not been run. Once an action is triggered, the Catalyst optimizer will review the execution plan, then optimize the entire stage, and then perform processing on the optimized plan.

1.3. The underlying of Spark vs. MapReduce

Spark DataFrames and MapReduce both enable distributed data processing but differ significantly in execution and efficiency. Both frameworks process large-scale datasets in parallel across clusters, breaking computations into smaller tasks that run on multiple nodes.

However, MapReduce follows a disk-based, batch processing model, where intermediate results are written to disk between the map and reduce phases, making it slower due to frequent I/O operations. In contrast, Spark DataFrames leverage in-memory processing, significantly reducing disk I/O and improving speed.

Another key difference is that MapReduce requires users to explicitly define map and reduce functions, whereas DataFrames provide a higher-level, SQL-like API that abstracts these details, allowing for more readable and concise code.

Additionally, Spark DataFrames benefit from Catalyst Optimizer and Tungsten Engine, which apply automatic query optimizations and memory-efficient execution, whereas MapReduce lacks built-in query optimization and relies solely on user-defined logic.

Despite these differences, both frameworks perform similar fundamental operations, such as mapping, shuffling, and reducing data, but Spark DataFrames streamline this process with optimized execution plans and a more user-friendly API, making them a more efficient and flexible alternative to MapReduce.

2. Problem Statements

In all exercises below, your team may select any programming language among Java, Scala, and Python to implement the solutions. Similar to the previous Lab, **only solutions written in Scala will be awarded the full programming language score** for each problem; solutions in Python or Java will not receive full points. Note that you are only allowed to fall back on the native code or libraries of that language if Apache Spark's built-in APIs are not sufficient to accomplish the required objectives. For example, Numpy is considered a native library if Python is your selected programming language.

In all exercises below, you will use the same Amazon Sale Report¹ dataset that is included in this Lab. The allowed environments are that you have already *installed* from Lab 1, other environments such as Google Colab are not permitted in this Lab. The benchmark, if any, should be run in at least 5 times and the resulted measurements shall be averaged across these runs, numerical means and standard deviations shall be included in the final report for your benchmarking results.

2.1. Advanced MapReduce problems

You will use the **MapReduce framework** to work on a couple of following problems:

1. Implement a **dynamic-length sliding window** computation that identifies the size that is mostly bought at each state within a variable number of days prior to the current date. The window length is determined as follows: first, compute the total number of "bought" orders per state across the **entire dataset**; if a state has **more than 10,000** total bought orders, the window length for that state is **5 days** (from $d-5$ to $d-1$); otherwise, the window length is **10 days** (from $d-10$ to $d-1$). The window length may be shorter if there are no appropriate past orders near the beginning of the dataset. An item is considered "bought" if the associated order has "shipped" in its status and the quantity is non-zero. The window should slide by 1 day at a time, thus unseen

¹ From [E-Commerce Sales Dataset](#), formatted as *asr.csv* and provided within the problemset's ZIP file

timestamps may arise in the result which is expected for a sliding window's result.

In the case where **two or more sizes share the same highest frequency** within a window, the following tie-breaking rules shall be applied in order:

- *First, select the size whose **variance of the purchased amount** (within that window) is **lower**. The variance is computed as the population variance (i.e., dividing by N , not $N-1$).*
- *If there is still a tie after comparing variances, select the size that is **lexicographically smallest** (e.g., "L" < "M" < "S" < "XL" < "XXL").*

Your team is expected to design the solution using a **map-to-buckets** approach where each record is mapped into all window buckets that it belongs to, followed by a combine/reduce phase per (state, window_date, size). This problem is centered around a concept called sliding window so you may need to review relevant knowledge prior to working on it.

Report requirements for this problem: Your team must provide a detailed analysis including: (a) the key design and how each record is assigned to window buckets, (b) the theoretical **time complexity** of the approach (e.g., how the naive $O(n \times w)$ can be optimized), (c) the **shuffle complexity** — how much data is transferred between the map and reduce phases, and (d) justification for the tie-breaking logic.

2. The "variety" of a style is defined as the number of distinct SKU associated with that style within a specific time interval and in a specific geographical region. Median variety is used to estimate the variety of goods purchased within this time-space interval and it is computed as the median value of all style that satisfies a specific condition. Now, for each month (for example, July starts at 07-01 and ends at 07-31), you are required to calculate the state-level median variety of all style which has served a size of least XXL (for example, XXL, 3XL, 4XL, .etc).

For each of the above problem, export the final result into a single CSV file that is readable under normal filesystem (rather than Hadoop-specific files). The filename is specified under grading criteria of this problemset's last section while the team is free to select the appropriate file's schema. Note that, for solutions written in Python, any streaming-based method for processing the queries is NOT permitted.

Report: Your team must provide detailed analysis on the requested queries, including but not limited to how the team understand and frame the queries, how do the team decompose them as well as the implementation strategy for each decomposed step.

2.2. Problems for Structured APIs

Using Apache Spark's **Structured APIs**, within which the Spark Dataset is allowed for Scala and/or Java solutions while Spark SQL is not accepted to be graded (though

Spark SQL can be used to illustrate your understanding of the queries, or as intermediate steps that lead to the final solutions):

1. For each city, calculate the percentage of cancelled orders of **Standard** service level that possess at least **3 temporally-valid promotions** while having the purchased amount less than the average amount of the associated state's merchant-fulfillment orders which have a courier status of "Shipped".

A promotion is considered **temporally valid** if its **active period** spans at least **2 days**. The active period of a promotion is derived from the dataset as follows: for each unique promotion identifier appearing across all orders, its **first appearance date** and **last appearance date** are computed; the active period is defined as the number of days between these two dates. All promotions including those issued by Amazon, are counted towards the criterion of 3 simultaneous ones, provided that they satisfy the temporal validity condition above.

Additional technical requirements for this problem:

- *Your solution must exclusively use the **DataFrame/Dataset API**. Direct Spark SQL string queries are not accepted.*
 - *Your team must include the output of **explain(true)** (or the equivalent extended execution plan) in the report.*
 - *The report must identify and analyze: (a) which **physical join strategy** Spark selects (e.g., **BroadcastHashJoin**, **SortMergeJoin**, **BroadcastNestedLoopJoin**), (b) the **number of shuffle exchanges** (Exchange nodes in the plan), and (c) the **number of stages** produced by the query.*
2. For each SKU within each month, compute the standard deviation of the amount of orders whose number of promotions meets a **dynamic percentile threshold**. Specifically, two percentile levels are required:
 - **P90** (the 90th percentile): *select all orders whose number of promotions is **at or above** the 90th percentile of promotion counts within that SKU-month group. Compute the population standard deviation (degree of freedom = 0) of the purchased amounts of these selected orders.*
 - **P80** (the 80th percentile): *apply the same logic using the 80th percentile as the threshold.*

The number of promotions for each order is determined by counting **all** promotion identifiers associated with that order (including Amazon-issued promotions). If a SKU-month group contains fewer than 2 qualifying orders after the percentile filter, the standard deviation is set to zero.

Your team must implement **two approaches** for computing the percentile thresholds:

- Using Spark's built-in ***approx_percentile*** (or ***percentile_approx***) function.
- A ***self-implemented exact percentile*** computation using *DataFrame/Dataset* operations.

The report must include a comparison of these two approaches in terms of: (a) **accuracy** — the difference between approximate and exact thresholds, (b) **execution time**, and (c) an analysis of any SKU-month groups where the two approaches yield **different sets of qualifying orders**.

Additionally, if any SKU-month group contains **more than 1,000 orders**, your team must discuss: (a) whether manual **repartitioning** is beneficial for that group, (b) the reasoning behind the chosen partition strategy, and (c) how Spark's default partition size (typically 128 MB) relates to the group's data volume.

For each of the above problem, export the final result into a single PARQUET file that can be parsed by either Pandas or Spark in local mode and under normal filesystem (rather than Hadoop-specific files). The filename is specified under grading criteria of this problemset's last section while the team is free to select the appropriate file's schema.

Report: Your team must provide detailed analysis on the requested queries, including but not limited to how the team understand and frame the queries, how do the team decompose them as well as the implementation strategy for each decomposed step.

3. Submission Guideline

This lab requires a group's submission where the work of your team's members is compressed into a single file and only one representative may submit this file on Moodle. Note that, different from Lab 1, Lab 2 requires the team to collaborate towards accomplishing the stated problems rather than the team's members working individually, hence the submitted solution should be a single, unified one. The submission file contains a single folder named *<RepresentativeID>* where student ID of any member of your team is used. This folder's internal structure contains the team's labwork for each problem. Specifically the structure will be as follow:

```
<RepresentativeID>
|--- src
|   |--- Task_1-1
|   |--- | source          # Code files here
|   |--- | ...
|   |--- Task_1-2
|   |--- | source          # Code files here
|   |--- | ...
```

```

|      |--- Task_2-1
|      |--- | source          # Code files here
|      |--- | ...
|      |--- Task_2-2
|      |--- | source          # Code files here
|      |--- | ...
|--- docs
|      |--- Report.pdf
|      |--- drive_link.txt
|      |--- README (optional, instructions to run the code)

```

The drive_link.txt contains a single link to Google Drive folder that is structurally organized as described below. Of course, any edit beyond the determined deadline published on Moodle will invalidate your team's result.

<RepresentativeID>

```

|--- Task_1-1.csv
|--- Task_1-2.csv
|--- Task_2-1.parquet
|--- Task_2-2.parquet

```

Grading Criteria

The grading criteria for each problem in the Problem Statements section are summarized in the below table with each problem contributing 2.5 points.

Requirements	Points
Each problem	2.5
- Correct analysis of the queries	0.5
- Successful decomposition into elemental steps	0.5
- Explaining the reasoning behind this decomposition	0.5
- Successful implementation of the above steps	0.5
- Tested and exported successfully	0.2
- Correctness of exported result	0.175
- The solution is in runnable Scala	0.125
TOTAL of 4 problems	10

You must strictly follow the aforementioned file structure and compress the whole folder into a ZIP file named <RepresentativeID>.zip, which is your final file to be submitted to Moodle.

- Ensure your code is well-documented with clear comments.
 - Each task can be accomplished under complex environments and different programming languages, remember to provide instructions for running each task if this is the case.
-

Happy Coding and Best of Luck!

The Lab Instructor./.